SRI RAMACHANDRA
INSTITUTE OF HIGHER EDUCATION AND RESEARCH
(Category - I Deemed to be University) Porur, Chennai
SRI RAMACHANDRA FACULTY OF ENGINEERING AND TECHNOLOGY

MED23CL202 · PYTHON PROGRAMMING LABORATORY

B.Tech Computer Science and Medical Engineering
(Artificial Intelligence and Data Analytics)
Year II / Semester III

MINI-PROJECT REPORT

VitalCheck: a ward vitals monitor that flags abnormal readings

Submitted by

Name	Priya S
Register number	E0125001
Batch	2025-2029
Academic year	2026–2027
Faculty in-charge	Ms. Aishwarya S

SRI RAMACHANDRA
INSTITUTE OF HIGHER EDUCATION AND RESEARCH
(Category - I Deemed to be University) Porur, Chennai
SRI RAMACHANDRA FACULTY OF ENGINEERING AND TECHNOLOGY

BONAFIDE CERTIFICATE

Name : Priya S
Register number : E0125001
Batch : 2025-2029
Academic year : 2026–2027

Certified that this is the bonafide record of the mini-project titled "**VitalCheck: a ward vitals monitor that flags abnormal readings**" carried out by the above student in the course **MED23CL202 – Python Programming Laboratory** during the academic year 2026–2027.

_____
Faculty in-charge
Ms. Aishwarya S

_____
Head of the Department
Dr. Nandakumar Venkatesan

Submitted for the Practical Examination held on _____

_____
Internal Examiner

_____
External Examiner

DECLARATION

I, Priya S (Reg. No. E0125001), declare that this mini-project report titled “VitalCheck: a ward vitals monitor that flags abnormal readings” is my own work, done as part of MED23CL202 – Python Programming Laboratory under the guidance of Ms. Aishwarya S. The program code and results in this report were written, run and tested by me. Any code, data or idea taken from another source is credited in the References.

Place: Chennai
Date: _____ _____
Signature of the student

ACKNOWLEDGEMENT

I thank my faculty in-charge and the lab staff for their guidance, and my classmates for testing VitalCheck.

SRI RAMACHANDRA
INSTITUTE OF HIGHER EDUCATION AND RESEARCH
(Category - I Deemed to be University) Porur, Chennai
SRI RAMACHANDRA FACULTY OF ENGINEERING AND TECHNOLOGY

PYTHON PROGRAMMING LABORATORY

MED23CL202 · Year II / Semester III

CERTIFICATE OF PROGRESS

e-Lab Record (Foothold)

This is to certify that

Priya S

Register number: E0125001

has completed **7 of the ten experiments** and **2 of the five mini-project checkpoints** of the e-Lab Record (9 of 15 activities, 60%), as part of the **Python Programming Laboratory (MED23CL202)**.

ACADEMIC YEAR 2026–2027

Date: **October 2026**

Ms. Aishwarya S
Faculty in-charge

Issued from the student's saved progress in Foothold when this report was downloaded.

CONTENTS

1. INTRODUCTION 6

1.1 Who uses it, and where 6

1.2 What my program does 6

1.3 The problem 6

2. OBJECTIVES 6

3. PYTHON CONCEPTS IN MY PROGRAM 6

3.1 Experiment 1: Data types, operators and decisions 7

3.2 Experiment 2: Loops, collections and functions 8

3.3 Experiment 3: Classes, objects and encapsulation 9

3.4 Experiment 4: Inheritance and polymorphism 11

3.5 Experiments 5 and 6: File handling 11

3.6 Experiment 7: Handling errors with try / except 12

3.7 Experiment 8: raise, finally and your own exception 13

3.8 Experiment 9: Data analysis with pandas 14

3.9 Experiment 10: Charts 14

4. PROGRAM RUNNING 16

5. TESTING 16

6. REFLECTION 16

6.1 The hardest bug 16

6.2 Limitations and safety 17

6.3 Conclusion 17

6.4 Future scope 17

7. REFERENCES 17

APPENDIX A: FULL PROGRAM CODE 18

1. INTRODUCTION

1.1 Who uses it, and where

Users: Nurse **Place:** Hospital ward

1.2 What my program does

VitalCheck lets a ward nurse record each patient's temperature, pulse and blood pressure. It flags abnormal readings at once, uses stricter limits for ICU patients, saves every reading in a file in Google Drive, and shows a summary and charts.

1.3 The problem

Problem today	How my program helps
Vital signs are written on a paper chart, so an abnormal reading is noticed only when someone reads the chart.	VitalCheck labels every reading NORMAL, CHECK or URGENT the moment it is entered.
ICU patients need closer watching, but one set of limits is used for everyone.	ICU patients get stricter limits, so a smaller change is flagged sooner.
Typing mistakes such as 160 °F end up in the record.	Impossible values are refused with a clear message, and nothing is saved.

2. OBJECTIVES

1. To store each patient's readings in a class, so they can only be added through a method that checks them.
2. To flag abnormal readings at once, with stricter limits for ICU patients.
3. To refuse impossible values and wrong input with a clear message instead of crashing.
4. To save every reading in a file so it is not lost when the program closes.
5. To summarise the readings with pandas and show them in charts.

3. PYTHON CONCEPTS IN MY PROGRAM

My program uses 25 of the 25 course concepts. Each one is shown below with its lines from my code. Lines marked “Checked by Foothold” were confirmed by Python’s own parser when this report was made; the full code, with every tagged line marked, is in Appendix A.

Exp	Code	Concept	Where in my code	Used
1	`IN`	Input with type conversion	vitalcheck.py lines 122–123	Yes
1	`AR`	Arithmetic operators	vitalcheck.py line 140	Yes

Exp	Code	Concept	Where in my code	Used
1	CO	Comparison and logical operators	vitalcheck.py line 57	Yes
1	IF	Decisions (if / elif / else)	vitalcheck.py lines 98–101	Yes
2	FO	for loop	vitalcheck.py lines 96–101	Yes
2	WH	while loop	vitalcheck.py lines 119–125	Yes
2	LI	List	vitalcheck.py line 95	Yes
2	DI	Dictionary	vitalcheck.py line 38	Yes
2	FN	Function that returns a value	vitalcheck.py lines 128–135	Yes
2	MO	Module (import)	vitalcheck.py lines 13–20	Yes
3	CL	Class	vitalcheck.py lines 64–102	Yes
3	OB	Objects created	vitalcheck.py line 294	Yes
3	PR	Private attribute with getter / setter	vitalcheck.py lines 70, 82, 84	Yes
4	SU	Inheritance (subclass)	vitalcheck.py line 105	Yes
4	OV	Overriding or polymorphism	vitalcheck.py lines 111–112	Yes
5–6	FI	File handling	vitalcheck.py lines 152–157	Yes
7	TR	try / except	vitalcheck.py lines 120–125	Yes
7	ME	More than one specific exception	vitalcheck.py lines 180–181	Yes
8	RA	raise	vitalcheck.py line 58	Yes
8	FY	finally	vitalcheck.py lines 160–162	Yes
8	EX	Your own (user-defined) exception	vitalcheck.py lines 49, 58, 309	Yes
9	PD	pandas DataFrame	vitalcheck.py line 192	Yes
9	FL	Select and filter data	vitalcheck.py lines 195–197	Yes
10	PL	Matplotlib chart with labels	vitalcheck.py lines 213–222	Yes
10	SB	Seaborn chart	vitalcheck.py line 228	Yes

3.1 Experiment 1: Data types, operators and decisions

IN Input with type conversion

vitalcheck.py, lines 122–123

```
122 IN                      return int(input(prompt))
```

```
123 IN                 return float(input(prompt))
```

Checked by Foothold: float(input(...)), int(input(...)).

AR Arithmetic operators

vitalcheck.py, line 140

```
140 AR         return round(celsius * 9 / 5 + 32, 1)
```

Checked by Foothold: arithmetic operators.

CO Comparison and logical operators

vitalcheck.py, line 57

```
57 CO         if value < low or value > high:
```

Checked by Foothold: comparison and logical operators.

IF Decisions (if / elif / else)

vitalcheck.py, lines 98–101

```
98 IF                 if value > high:
99 IF                     alerts.append(f"{name} HIGH ({value} {UNITS[name]})")
100 IF                elif value < low:
101 IF                    alerts.append(f"{name} LOW ({value} {UNITS[name]})")
```

Checked by Foothold: if.

3.2 Experiment 2: Loops, collections and functions

FO for loop

vitalcheck.py, lines 96–101

```
96 FO             for name, (low, high) in self.limits().items():
97 FO                 value = reading[name]
98 FO                 if value > high:
99 FO                     alerts.append(f"{name} HIGH ({value} {UNITS[name]})")
100 FO                elif value < low:
101 FO                    alerts.append(f"{name} LOW ({value} {UNITS[name]})")
```

Checked by Foothold: for.

WH while loop

vitalcheck.py, lines 119–125

```
119 WH         while True:
120 WH             try:
121 WH                 if kind == int:
122 WH                     return int(input(prompt))
123 WH                 return float(input(prompt))
124 WH             except ValueError:
125 WH                 print("  Please type a number, e.g. 98.6")
```

Checked by Foothold: while.

LI List

vitalcheck.py, line 95

```
95 LI             alerts = []
```

Checked by Foothold: list.

DI Dictionary

vitalcheck.py, line 38

```
38 DI     POSSIBLE = {"temp": (85.0, 115.0), "pulse": (20, 250), "systolic": (50, 260)}
```

Checked by Foothold: dictionary.

FN Function that returns a value

vitalcheck.py, lines 128–135

```
128 FN     def alert_label(alerts):
129 FN         """Turn a list of alerts into one short label."""
130 FN         if not alerts:
131 FN             return "NORMAL"
132 FN         elif len(alerts) == 1:
133 FN             return "CHECK: " + alerts[0]
134 FN         else:
135 FN             return "URGENT: " + "; ".join(alerts)
```

Checked by Foothold: alert_label.

MO Module (import)

vitalcheck.py, lines 13–20

```
13 MO     import os
14 MO     import csv
15 MO     import statistics
16 MO     from datetime import datetime
17 MO
18 MO     import pandas as pd
19 MO     import matplotlib.pyplot as plt
20 MO     import seaborn as sns
```

Checked by Foothold: os, csv.

3.3 Experiment 3: Classes, objects and encapsulation

CL Class

vitalcheck.py, lines 64–102 (first 30 lines shown)

```
64 CL     class Patient:
65 CL         """A patient on the ward and their vital-sign readings."""
66 CL
67 CL         def __init__(self, name, bed):
68 CL             self.name = name
```

```
69 CL             self.bed = bed
70 CL             self.__readings = []          # private: changed only through add_reading()
71 CL
72 CL         def add_reading(self, temp, pulse, systolic, time=None):
73 CL             """Check a reading, then store it. Raises InvalidReadingError for impossible
       values."""
74 CL             check_possible("temp", temp)
75 CL             check_possible("pulse", pulse)
76 CL             check_possible("systolic", systolic)
77 CL             reading = {"time": time or datetime.now().strftime("%Y-%m-%d %H:%M"),
78 CL                        "temp": temp, "pulse": pulse, "systolic": systolic}
79 CL             self.__readings.append(reading)
80 CL             return reading
81 CL
82 CL         def get_readings(self):
83 CL             """Return a copy, so outside code can't change the stored readings."""
84 CL             return list(self.__readings)
85 CL
86 CL         def kind(self):
87 CL             return "Ward"
88 CL
89 CL         def limits(self):
90 CL             """The normal limits used for this patient."""
91 CL             return NORMAL
92 CL
93 CL         def status(self, reading):
```

Checked by Foothold: Patient.

OB Objects created

vitalcheck.py, line 294

```
294 OB                 patients[name] = ICUPatient(name, bed) if icu else Patient(name, bed)
```

Checked by Foothold: ICUPatient(...), Patient(...).

PR Private attribute with getter / setter

vitalcheck.py, line 70

```
70 PR             self.__readings = []          # private: changed only through add_reading()
```

vitalcheck.py, line 82

```
82 PR         def get_readings(self):
```

vitalcheck.py, line 84

```
84 PR             return list(self.__readings)
```

Checked by Foothold: get_readings, __readings.

3.4 Experiment 4: Inheritance and polymorphism

SU Inheritance (subclass)

vitalcheck.py, line 105

```
105 SU     class ICUPatient(Patient):
```

Checked by Foothold: ICUPatient(Patient).

OV Overriding or polymorphism

vitalcheck.py, lines 111–112

```
111 OV         def limits(self):                # overrides Patient.limits()
112 OV             return ICU_NORMAL
```

Checked by Foothold: ICUPatient.limits overrides Patient.limits.

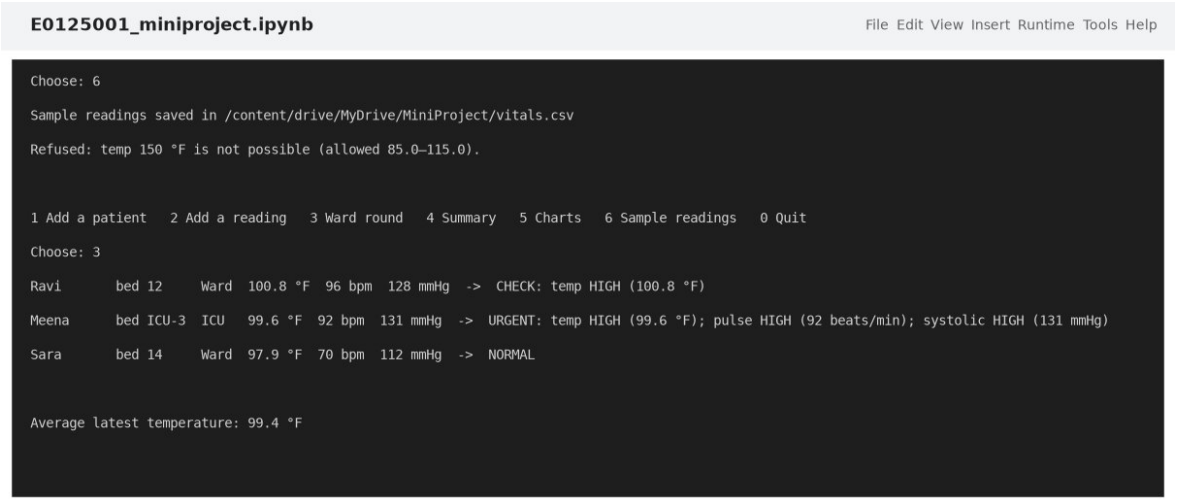


Figure 1: Ward round: Ravi is flagged at 100.8 °F, while Meena (ICU) is already flagged at 99.6 °F

3.5 Experiments 5 and 6: File handling

FI File handling

vitalcheck.py, lines 152–157

```
152 FI             with open(DATA_FILE, "a", newline="") as f:
153 FI                 writer = csv.writer(f)
154 FI                 if new_file:
155 FI                     writer.writerow(FIELDS)
156 FI                 writer.writerow([reading["time"], patient.name, patient.bed, patient.kind(),
157 FI                                  reading["temp"], reading["pulse"], reading["systolic"]])
```

Checked by Foothold: with open, open.

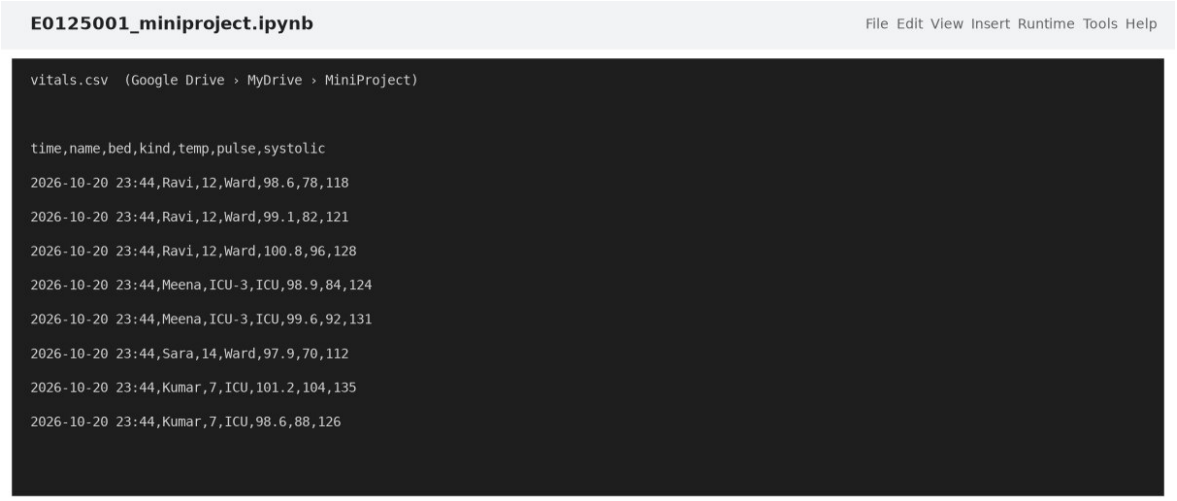


Figure 2: Data saved in the file

My program saves its file in: Google Drive

3.6 Experiment 7: Handling errors with try / except

TR try / except

vitalcheck.py, lines 120–125

```
120 TR            try:
121 TR                if kind == int:
122 TR                    return int(input(prompt))
123 TR                return float(input(prompt))
124 TR            except ValueError:
125 TR                print("  Please type a number, e.g. 98.6")
```

Checked by Foothold: try.

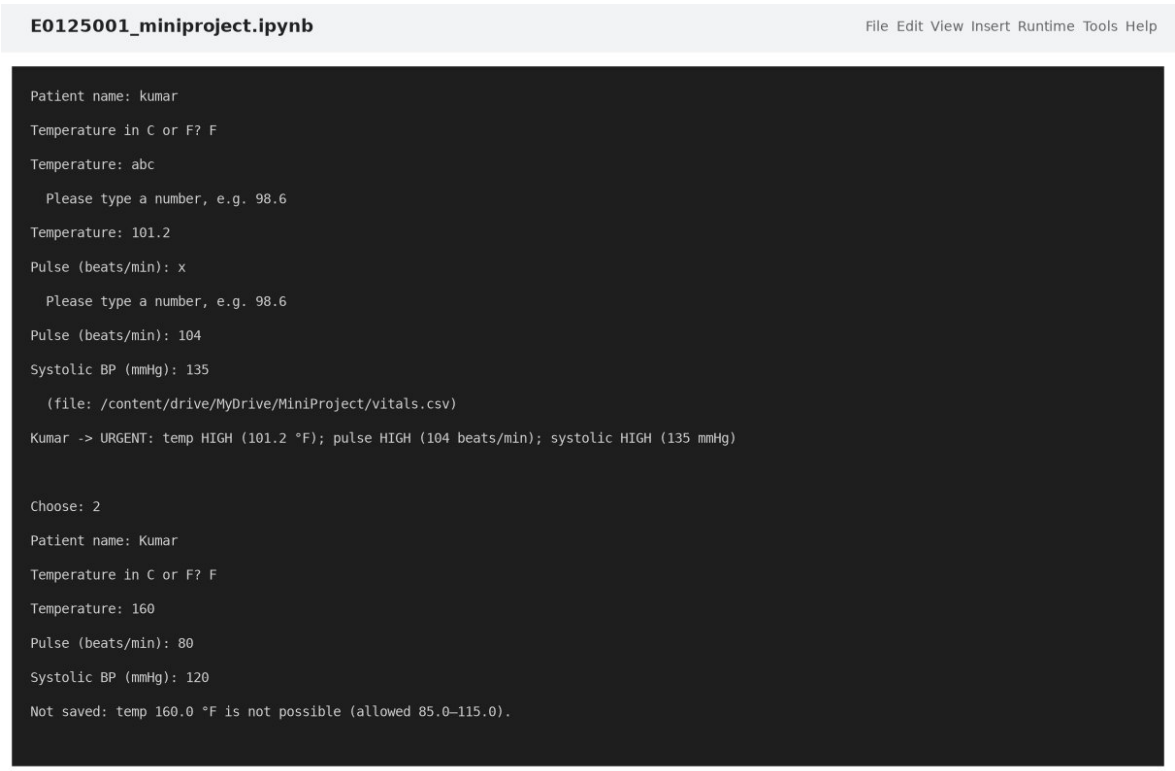


Figure 3: Wrong input handled with a message (no crash)

ME More than one specific exception

vitalcheck.py, lines 180–181

```
180 ME                  except (ValueError, InvalidReadingError):
181 ME                      print("  Skipped a damaged row:", row)
```

Checked by Foothold: 2 exception types.

3.7 Experiment 8: raise, finally and your own exception

RA raise

vitalcheck.py, line 58

```
 58 RA              raise InvalidReadingError(f"{name} {value} {UNITS[name]} is not possible (allowed
        {low}–{high}).")
```

Checked by Foothold: raise.

FY finally

vitalcheck.py, lines 160–162

```
160 FY          finally:
161 FY              if not quiet:
162 FY                  print(f"  (file: {DATA_FILE})")
```

Checked by Foothold: finally.

EX Your own (user-defined) exception

vitalcheck.py, line 49

```
 49 EX      class InvalidReadingError(Exception):
```

vitalcheck.py, line 58

```
58 EX             raise InvalidReadingError(f"{name} {value} {UNITS[name]} is not possible (allowed
       {low}–{high}).")
```

vitalcheck.py, line 309

```
309 EX                except InvalidReadingError as e:
```

Checked by Foothold: defined, raised, caught.

See Figure 3: Wrong input handled with a message (no crash).

3.8 Experiment 9: Data analysis with pandas

PD pandas DataFrame

vitalcheck.py, line 192

```
192 PD         df = pd.read_csv(DATA_FILE)
```

Checked by Foothold: pd.read_csv.

FL Select and filter data

vitalcheck.py, lines 195–197

```
195 FL         table = df.groupby("name")[["temp", "pulse", "systolic"]].agg(["mean", "min",
       "max"]).round(1)
196 FL         print(table)
197 FL         fevers = df[df["temp"] >= 100.4]
```

Checked by Foothold: selects data, agg().

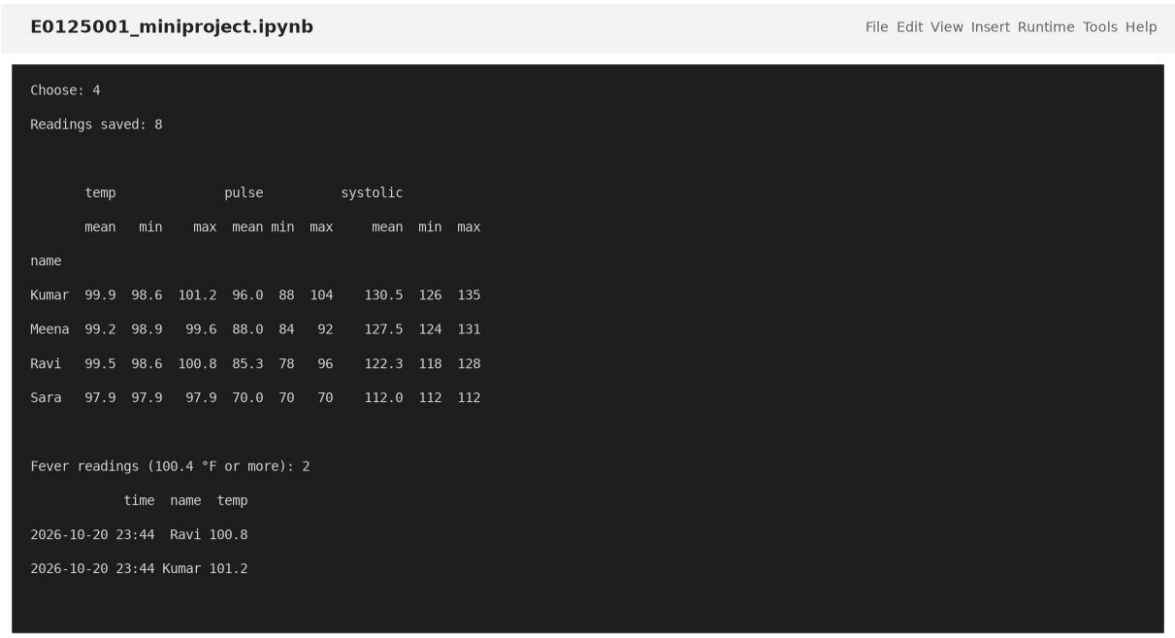


Figure 4: pandas summary of the data

3.9 Experiment 10: Charts

PL Matplotlib chart with labels

vitalcheck.py, lines 213–222

```
213 PL         plt.figure(figsize=(7, 3.5))
```

```
214 PL        plt.plot(range(1, len(rows) + 1), rows["temp"], marker="o")
215 PL        plt.axhline(100.4, color="red", linestyle="--", label="Fever (100.4 °F)")
216 PL        plt.xticks(range(1, len(rows) + 1))
217 PL        plt.xlabel("Reading number")
218 PL        plt.ylabel("Temperature (°F)")
219 PL        plt.title(f"Temperature of {name}")
220 PL        plt.legend()
221 PL        plt.tight_layout()
222 PL        plt.show()
```

Checked by Foothold: plt.figure, plt.plot.

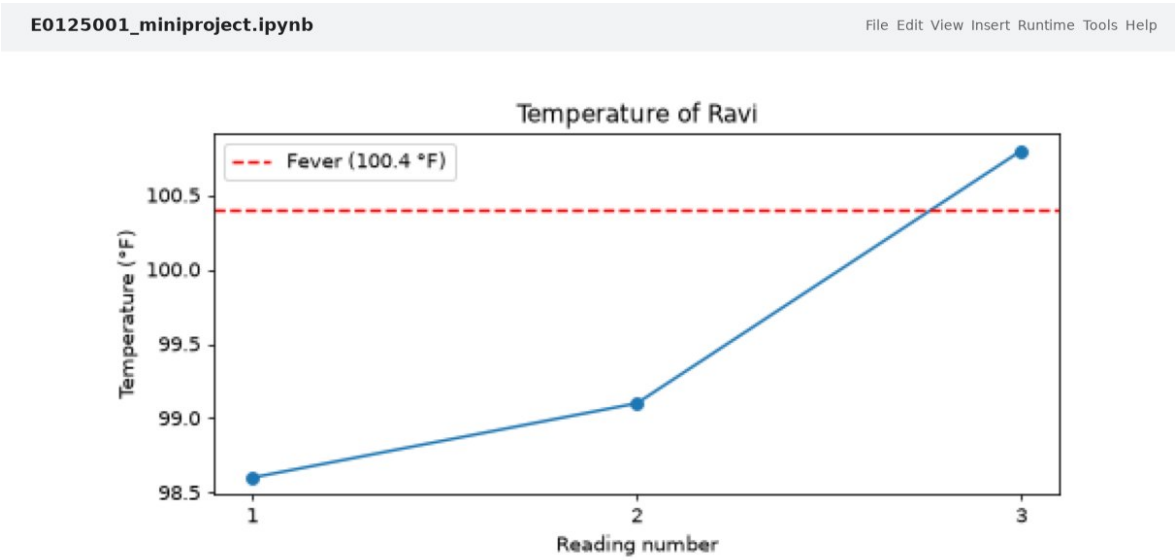


Figure 5: Temperature of Ravi, with the fever line (Matplotlib)

SB Seaborn chart

vitalcheck.py, line 228

```
228 SB        sns.histplot(data=df, x="temp", bins=8)
```

Checked by Foothold: sns.histplot.

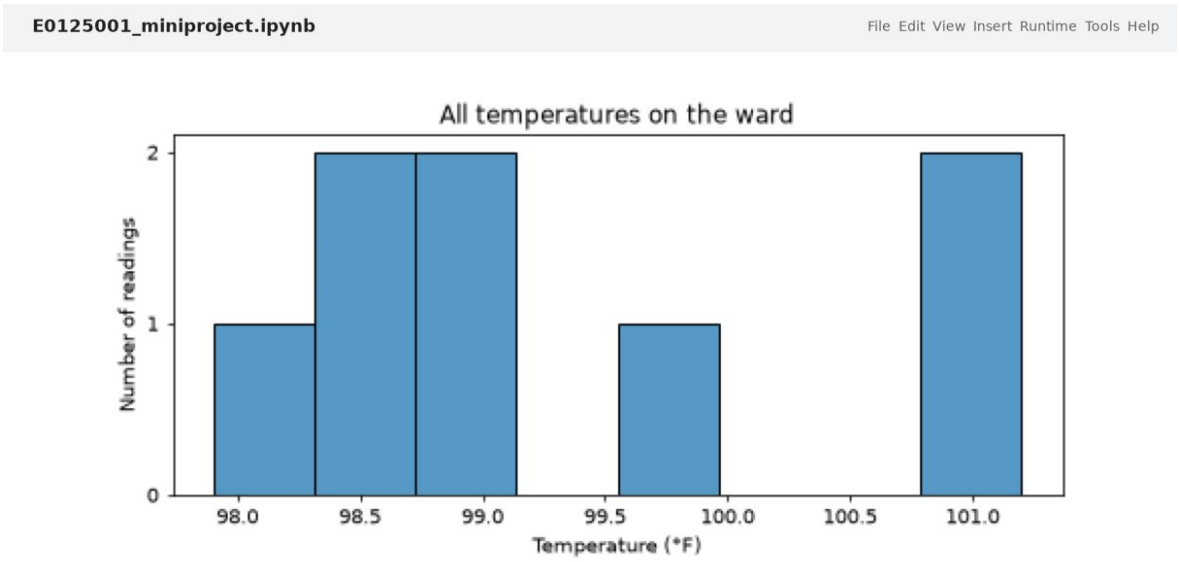


Figure 6: All temperatures on the ward (Seaborn histogram)

4. PROGRAM RUNNING

All screenshots appear in Section 3, beside the concepts they show.

5. TESTING

No.	Type	What I tried	What happened	Result
1	Wrong type	Typed “abc” for the temperature	It said “Please type a number” and asked again	**Pass**
2	Impossible value	Temperature 160 °F	“temp 160.0 °F is not possible (allowed 85.0–115.0)”, not saved	**Pass**
3	Boundary value	ICU patient, 99.5 °F	CHECK: temp HIGH (99.5 °F), stricter ICU limit	**Pass**
4	Missing or empty file	First run, before any reading is saved	“0 patient(s) loaded”; summary says “No readings saved yet.”	**Pass**

4 tests recorded; 4 passed, 0 failed.

6. REFLECTION

6.1 The hardest bug

What went wrong	The chart for "ravi" said there were no readings, although Ravi's readings were in the file.
How I found the cause	I printed the names column and saw the names were saved exactly as typed, "Ravi" and "ravi".

How I fixed it	I used .strip().title() on every name the user types, so "ravi" and "Ravi" become the same patient.

6.2 Limitations and safety

One set of adult limits is used for every ward patient, whatever their age. Anyone with the notebook can read the data file. The limits are simplified teaching values, not clinical guidelines.

This program is a student learning project. It is not a medical device and must not be used to make decisions about real patients.

6.3 Conclusion

VitalCheck records temperature, pulse and blood pressure, flags abnormal readings at once with stricter ICU limits, refuses impossible values, and keeps every reading in a file for the summary and charts. All five objectives were met. I learned how classes, my own exception and file handling work together in one program.

6.4 Future scope

- Add a time chart for pulse and BP too.
- Warn when a patient's temperature rises in three readings in a row.

Use of AI tools: This sample was written with help from Claude, an AI assistant.

7. REFERENCES

[1] Python Software Foundation, “The Python Tutorial,” docs.python.org/3/tutorial.

[2] pandas development team, “pandas documentation,” pandas.pydata.org.

[3] Matplotlib and seaborn documentation, matplotlib.org and seaborn.pydata.org.

[4] MED23CL202 Python Lab Manual and the Foothold Python course (Experiments 1–10).

APPENDIX A: FULL PROGRAM CODE

Every line I tagged is marked with a coloured bar and the concept's letter code:

IN Input with type conversion **AR** Arithmetic operators **CO** Comparison and logical operators **IF** Decisions (if / elif / else) **FO** for loop **WH** while loop **LI** List **DI** Dictionary **FN** Function that returns a value **MO** Module (import) **CL** Class **OB** Objects created **PR** Private attribute with getter / setter **SU** Inheritance (subclass) **OV** Overriding or polymorphism **FI** File handling **TR** try / except **ME** More than one specific exception **RA** raise **FY** finally **EX** Your own (user-defined) exception **PD** pandas DataFrame **FL** Select and filter data **PL** Matplotlib chart with labels **SB** Seaborn chart

vitalcheck.py

```
1         # VitalCheck: a ward vitals monitor
2         # Sample mini-project, Track A (Patient Vitals Monitor), MED23CL202 Python Programming
          Laboratory
3         #
4         # A nurse records each patient's temperature, pulse and blood pressure. VitalCheck flags
5         # abnormal readings at once, uses stricter limits for ICU patients, saves every reading
6         # in Google Drive, and shows a summary and charts.
7         # The normal limits are simplified teaching values. This is a student project, not a
          medical device.
8
9
10
11        # ── 1. Modules, and where the data file lives (Experiments 2, 5, 6) ──
12
13  MO    import os
14  MO    import csv
15  MO    import statistics
16  MO    from datetime import datetime
17  MO
18  MO    import pandas as pd
19  MO    import matplotlib.pyplot as plt
20  MO    import seaborn as sns
21
22        try:
23            from google.colab import drive
24            drive.mount("/content/drive")
25            DATA_DIR = "/content/drive/MyDrive/MiniProject"      # permanent: Google Drive
26        except Exception:
27            DATA_DIR = "vitals_data"                              # not in Colab, or Drive not
          connected
28            print("Google Drive is not connected; saving in a local folder instead.")
29
30        os.makedirs(DATA_DIR, exist_ok=True)
31        DATA_FILE = os.path.join(DATA_DIR, "vitals.csv")
```

```
32          print("Readings are saved in:", DATA_FILE)
33
34
35          # ── 2. Limits (dictionaries) and my own exception (Experiments 1, 2, 8) ──
36
37          # Values outside these are impossible, so they are rejected as typing mistakes.
38  DI      POSSIBLE = {"temp": (85.0, 115.0), "pulse": (20, 250), "systolic": (50, 260)}
39
40          # Simplified normal limits for an adult (teaching values, not clinical advice).
41          NORMAL = {"temp": (97.0, 100.3), "pulse": (60, 100), "systolic": (90, 139)}
42
43          # Stricter limits for ICU patients: smaller changes need attention sooner.
44          ICU_NORMAL = {"temp": (97.0, 99.4), "pulse": (60, 90), "systolic": (100, 129)}
45
46          UNITS = {"temp": "°F", "pulse": "beats/min", "systolic": "mmHg"}
47
48
49  EX      class InvalidReadingError(Exception):
50              """Raised when a vital sign is outside what any living patient can have."""
51              pass
52
53
54          def check_possible(name, value):
55              """Raise InvalidReadingError if a value is impossible; otherwise return it."""
56              low, high = POSSIBLE[name]
57  CO          if value < low or value > high:
58  RA EX           raise InvalidReadingError(f"{name} {value} {UNITS[name]} is not possible (allowed
    {low}–{high}).")
59              return value
60
61
62          # ── 3. Classes: encapsulation, inheritance, overriding (Experiments 3, 4) ──
63
64  CL      class Patient:
65  CL          """A patient on the ward and their vital-sign readings."""
66  CL
67  CL          def __init__(self, name, bed):
68  CL              self.name = name
69  CL              self.bed = bed
70  CL PR           self.__readings = []          # private: changed only through add_reading()
71  CL
72  CL          def add_reading(self, temp, pulse, systolic, time=None):
```

```
73  CL              """Check a reading, then store it. Raises InvalidReadingError for impossible
        values."""
74  CL              check_possible("temp", temp)
75  CL              check_possible("pulse", pulse)
76  CL              check_possible("systolic", systolic)
77  CL              reading = {"time": time or datetime.now().strftime("%Y-%m-%d %H:%M"),
78  CL                         "temp": temp, "pulse": pulse, "systolic": systolic}
79  CL              self.__readings.append(reading)
80  CL              return reading
81  CL
82  CL PR       def get_readings(self):
83  CL              """Return a copy, so outside code can't change the stored readings."""
84  CL PR           return list(self.__readings)
85  CL
86  CL          def kind(self):
87  CL              return "Ward"
88  CL
89  CL          def limits(self):
90  CL              """The normal limits used for this patient."""
91  CL              return NORMAL
92  CL
93  CL          def status(self, reading):
94  CL              """Return a list of alerts for one reading (an empty list means all normal)."""
95  LI CL           alerts = []
96  FO CL           for name, (low, high) in self.limits().items():
97  FO CL               value = reading[name]
98  IF FO               if value > high:
    CL
99  IF FO                   alerts.append(f"{name} HIGH ({value} {UNITS[name]})")
    CL
100 IF FO               elif value < low:
    CL
101 IF FO                   alerts.append(f"{name} LOW ({value} {UNITS[name]})")
    CL
102 CL              return alerts
103
104
105 SU      class ICUPatient(Patient):
106             """An ICU patient: the same checks, with stricter limits."""
107
108             def kind(self):
109                 return "ICU"
110
111 OV          def limits(self):                 # overrides Patient.limits()
112 OV              return ICU_NORMAL
113
```

```
114
115         # ── 4. Safe input and helper functions (Experiments 1, 2, 7) ──
116
117         def read_number(prompt, kind=float):
118             """Ask until the user types a number; return it as an int or a float."""
119 WH          while True:
120 WH TR           try:
121 WH TR               if kind == int:
122 IN WH TR                 return int(input(prompt))
123 IN WH TR               return float(input(prompt))
124 WH TR           except ValueError:
125 WH TR               print("  Please type a number, e.g. 98.6")
126
127
128 FN      def alert_label(alerts):
129 FN          """Turn a list of alerts into one short label."""
130 FN          if not alerts:
131 FN              return "NORMAL"
132 FN          elif len(alerts) == 1:
133 FN              return "CHECK: " + alerts[0]
134 FN          else:
135 FN              return "URGENT: " + "; ".join(alerts)
136
137
138         def fahrenheit(celsius):
139             """Convert a Celsius thermometer reading to °F."""
140 AR          return round(celsius * 9 / 5 + 32, 1)
141
142
143         # ── 5. Saving and loading the readings file (Experiments 5, 6) ──
144
145         FIELDS = ["time", "name", "bed", "kind", "temp", "pulse", "systolic"]
146
147
148         def save_reading(patient, reading, quiet=False):
149             """Append one reading to the CSV file, writing the header the first time."""
150             new_file = not os.path.exists(DATA_FILE)
151             try:
152 FI              with open(DATA_FILE, "a", newline="") as f:
153 FI                  writer = csv.writer(f)
154 FI                  if new_file:
```

```
155 FI                  writer.writerow(FIELDS)
156 FI              writer.writerow([reading["time"], patient.name, patient.bed, patient.kind(),
157 FI                               reading["temp"], reading["pulse"], reading["systolic"]])
158             except OSError as e:
159                 print("Could not save the reading:", e)
160 FY          finally:
161 FY              if not quiet:
162 FY                  print(f"  (file: {DATA_FILE})")
163
164
165         def load_patients():
166             """Read the CSV file and rebuild every patient with their readings."""
167             patients = {}
168             if not os.path.exists(DATA_FILE):
169                 return patients
170             with open(DATA_FILE, "r", newline="") as f:
171                 reader = csv.reader(f)
172                 next(reader)                                # skip the header row
173                 for row in reader:
174                     time, name, bed, kind, temp, pulse, systolic = row
175                     if name not in patients:
176                         cls = ICUPatient if kind == "ICU" else Patient
177                         patients[name] = cls(name, bed)
178                     try:
179                         patients[name].add_reading(float(temp), int(pulse), int(systolic), time)
180 ME                  except (ValueError, InvalidReadingError):
181 ME                      print("  Skipped a damaged row:", row)
182             return patients
183
184
185         # ── 6. A pandas summary of all readings (Experiment 9) ──
186
187         def summary():
188             """Print each patient's average, lowest and highest readings, and every fever."""
189             if not os.path.exists(DATA_FILE):
190                 print("No readings saved yet.")
191                 return None
192 PD          df = pd.read_csv(DATA_FILE)
193             print("Readings saved:", df.shape[0])
194             print()
195 FL          table = df.groupby("name")[["temp", "pulse", "systolic"]].agg(["mean", "min",
        "max"]).round(1)
```

```
196 FL          print(table)
197 FL          fevers = df[df["temp"] >= 100.4]
198             print()
199             print(f"Fever readings (100.4 °F or more): {len(fevers)}")
200             if len(fevers) > 0:
201                 print(fevers[["time", "name", "temp"]].to_string(index=False))
202             return df
203
204
205         # ── 7. Charts: Matplotlib and Seaborn (Experiment 10) ──
206
207         def chart_patient(df, name):
208             """Line chart of one patient's temperature over time."""
209             rows = df[df["name"] == name]
210             if rows.empty:
211                 print("No readings for", name)
212                 return
213 PL          plt.figure(figsize=(7, 3.5))
214 PL          plt.plot(range(1, len(rows) + 1), rows["temp"], marker="o")
215 PL          plt.axhline(100.4, color="red", linestyle="--", label="Fever (100.4 °F)")
216 PL          plt.xticks(range(1, len(rows) + 1))
217 PL          plt.xlabel("Reading number")
218 PL          plt.ylabel("Temperature (°F)")
219 PL          plt.title(f"Temperature of {name}")
220 PL          plt.legend()
221 PL          plt.tight_layout()
222 PL          plt.show()
223
224
225         def chart_all(df):
226             """Histogram of every temperature on the ward."""
227             plt.figure(figsize=(7, 3.5))
228 SB          sns.histplot(data=df, x="temp", bins=8)
229             plt.gca().yaxis.get_major_locator().set_params(integer=True)
230             plt.xlabel("Temperature (°F)")
231             plt.ylabel("Number of readings")
232             plt.title("All temperatures on the ward")
233             plt.tight_layout()
234             plt.show()
235
236
237         # ── 8. The ward round: one call, every kind of patient (polymorphism) ──
```

```
238
239         def ward_round(patients):
240             """Print the latest reading and alert label for every patient."""
241             if not patients:
242                 print("No patients yet.")
243                 return
244             temps = []
245             for p in patients.values():
246                 readings = p.get_readings()
247                 if not readings:
248                     print(f"{p.name:10} bed {p.bed:6} {p.kind():4}  no readings")
249                     continue
250                 last = readings[-1]
251                 temps.append(last["temp"])
252                 print(f"{p.name:10} bed {p.bed:6} {p.kind():4}  {last['temp']} °F  {last['pulse']}
        bpm  "
253                       f"{last['systolic']} mmHg  ->  {alert_label(p.status(last))}")
254             if temps:
255                 print(f"\nAverage latest temperature: {statistics.mean(temps):.1f} °F")
256
257
258         # ── 9. Sample readings, so the program can be tried without typing ──
259
260         def add_sample_readings(patients):
261             """Add a few readings for three patients, and show one impossible value being
        refused."""
262             samples = [
263                 (Patient, "Ravi", "12", [(98.6, 78, 118), (99.1, 82, 121), (100.8, 96, 128)]),
264                 (ICUPatient, "Meena", "ICU-3", [(98.9, 84, 124), (99.6, 92, 131)]),
265                 (Patient, "Sara", "14", [(97.9, 70, 112)]),
266             ]
267             for cls, name, bed, readings in samples:
268                 if name not in patients:
269                     patients[name] = cls(name, bed)
270                 for temp, pulse, systolic in readings:
271                     reading = patients[name].add_reading(temp, pulse, systolic)
272                     save_reading(patients[name], reading, quiet=True)
273             print("Sample readings saved in", DATA_FILE)
274             try:
275                 patients["Sara"].add_reading(150, 70, 112)
276             except InvalidReadingError as e:
277                 print("Refused:", e)
278
```

```
279
280         # ── 10. The menu program ──
281
282         # `main` is just a name we chose: it is an ordinary function (Module 8) that holds the menu
            program.
283         # Defining it does not run it; the last line, main(), calls it and starts the menu.
284         def main():
285             patients = load_patients()
286             print(f"VitalCheck: {len(patients)} patient(s) loaded from the file.")
287             while True:
288                 print("\n1 Add a patient   2 Add a reading   3 Ward round   4 Summary   5 Charts
            6 Sample readings   0 Quit")
289                 choice = input("Choose: ").strip()
290                 if choice == "1":
291                     name = input("Patient name: ").strip().title()
292                     bed = input("Bed: ").strip()
293                     icu = input("ICU patient? (y/n): ").strip().lower() == "y"
294 OB                  patients[name] = ICUPatient(name, bed) if icu else Patient(name, bed)
295                     print(f"Added {name} ({patients[name].kind()}).")
296                 elif choice == "2":
297                     name = input("Patient name: ").strip().title()
298                     if name not in patients:
299                         print("No such patient. Add them first (option 1).")
300                         continue
301                     unit = input("Temperature in C or F? ").strip().upper()
302                     temp = read_number("Temperature: ")
303                     if unit == "C":
304                         temp = fahrenheit(temp)
305                     pulse = read_number("Pulse (beats/min): ", int)
306                     systolic = read_number("Systolic BP (mmHg): ", int)
307                     try:
308                         reading = patients[name].add_reading(temp, pulse, systolic)
309 EX                  except InvalidReadingError as e:
310                         print("Not saved:", e)
311                     else:
312                         save_reading(patients[name], reading)
313                         print(name, "->", alert_label(patients[name].status(reading)))
314                 elif choice == "3":
315                     ward_round(patients)
316                 elif choice == "4":
317                     summary()
318                 elif choice == "5":
319                     df = summary()
```

```
                    if df is not None:
                        chart_patient(df, input("Chart for which patient? ").strip().title())
                        chart_all(df)
                elif choice == "6":
                    add_sample_readings(patients)
                elif choice == "0":
                    print("Goodbye. All readings are saved in", DATA_FILE)
                    break
                else:
                    print("Please choose 0 to 6.")


        main()
```